

# Multimodal Deep Learning

## Exercise Sheet 3 (Week 3 and 4)

Date: 20th of May, 2025

### Exercise 1: Checking permutation-equivariance

Let  $\mathcal{S}_n$  be the permutation group on  $n$  indices and  $P_\pi \in \{0, 1\}^{n \times n}$  the permutation matrix for permutation  $\pi \in \mathcal{S}_n$ . I.e. the permutation matrix is a square binary matrix that describes  $\pi$ , has exactly one entry of 1 in each row and each column, and 0s elsewhere. A function  $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$  is *permutation-equivariant* iff

$$f(P_\pi x) = P_\pi f(x) \quad \forall x \in \mathbb{R}^n, \forall \pi \in \mathcal{S}_n.$$

For each of the following functions decide whether it is permutation-equivariant. Provide a short proof or a concrete counter-example (e.g. by exhibiting one  $x \in \mathbb{R}^n$  and one non-trivial permutation  $\pi$  that violates the condition).

Throughout, let  $x = (x_1, \dots, x_n)^\top$  with  $n \geq 3$ .

- (a) **Identity**  $f_{\text{id}}(x) = x$ .
- (b) **Element-wise non-linearity**  $f_{\text{ReLU}}(x) = (\max\{0, x_1\}, \dots, \max\{0, x_n\})^\top$ .
- (c) **Softmax vector**  $f_{\text{soft}}(x) = \left( \frac{e^{x_1}}{\sum_{j=1}^n e^{x_j}}, \dots, \frac{e^{x_n}}{\sum_{j=1}^n e^{x_j}} \right)^\top$ .
- (d) **Replicate first coordinate**  $f_{\text{first}}(x) = (x_1, x_1, \dots, x_1)^\top$ .
- (e) **Sorted output**  $f_{\text{sort}}(x) = \text{sort}(x)$  (returns the coordinates in ascending order).
- (f) **Linear layer with shared scalar weight**  $f_w(x) = w x$  for a fixed scalar  $w \in \mathbb{R}$ .
- (g) **Linear layer with dense weight matrix**  $f_W(x) = Wx$  for a fixed *arbitrary*  $W \in \mathbb{R}^{n \times n}$ .

#### Hints.

- The definition demands *all* permutations. It suffices to find a single counter-example  $(x, \pi)$  to disprove equivariance.
- For (g) you may want to compare  $W$  with  $P_\pi W P_\pi^\top$ .

### Exercise 2: Attention Masks

Attention masks can be represented in two main ways:

- **Binary attention masks:**  $M \in \{0, 1\}^{n \times n}$ , where  $M_{ij} = 1$  if token  $i$  (query) is allowed to attend to token  $j$  (key), and  $M_{ij} = 0$  otherwise. These are often used to multiply the raw attention scores before softmax, effectively zeroing out scores for disallowed pairs, or to directly prevent computation for these pairs.
- **Additive attention masks:** Here, the mask values are added to the attention logits (scores before softmax). Positions where attention is allowed are assigned a value of 0, while disallowed positions are assigned a large negative value (e.g.,  $-\infty$  or a very small number like  $-10^9$ ). This ensures that after the softmax operation, the probabilities for disallowed positions become zero.

In Parts (a)-(c) of this exercise, you will design *binary attention masks*. In Part (d), you will design an *additive attention mask*. For all binary masks, use the convention  $M_{ij} = 1 \iff$  token  $i$  is allowed to attend to token  $j$ .

(a) **Causal sequence (autoregressive) mask**

Consider a sequence of length  $n = 8$  processed left-to-right.

1. Write down the  $8 \times 8$  binary causal mask  $M^{\text{causal}}$  that allows each position  $i$  to attend only to positions  $k \leq i$  (including itself).
2. Give a closed-form expression for  $M_{ij}^{\text{causal}}$  (for a binary mask) for general sequence length  $n$ .

(b) **Binary tree mask**

A binary tree with seven nodes is stored in an array using breadth-first order (BFS), i.e., the tokens are [node0, node1, ..., node6] corresponding to [root, L, R, LL, LR, RL, RR] where L/R denote left/right children. The tree structure is shown in Figure 1. Design a binary mask  $M^{\text{tree}} \in \{0, 1\}^{7 \times 7}$  that fulfils *all* of the following conditions:

- A node may attend to its parent.
- A node may attend to its two children (if they exist).
- A node may also attend to itself.
- No other attention is allowed.

Write down the  $7 \times 7$  matrix  $M^{\text{tree}}$ .

Optionally, derive a general formula for  $M_{ij}^{\text{tree}}$  using integer arithmetic on  $i, j$  for a complete binary tree of  $N$  nodes stored in BFS order.

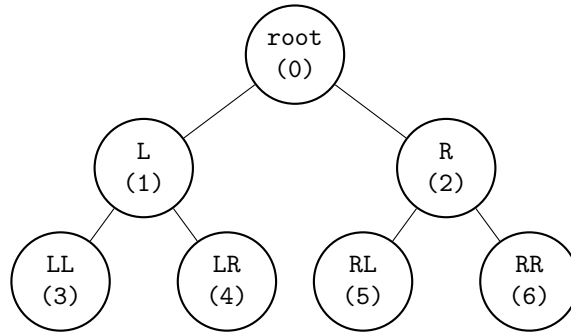


Figure 1: Binary tree structure for Part (b). Nodes are indexed 0-6 in Breadth-First Order (BFS). 'L' denotes left child, 'R' denotes right child.

(c) **Special datatype: heterogeneous tabular row**

A single table row is tokenised into a sequence of length  $1 + d_n + d_c$ :

$$[\text{CLS}, x_1^{\text{num}}, \dots, x_{d_n}^{\text{num}}, x_1^{\text{cat}}, \dots, x_{d_c}^{\text{cat}}],$$

where the first token **CLS** is a learned global representation, the  $d_n$  numeric features are  $x_i^{\text{num}}$ , and the  $d_c$  categorical features are  $x_i^{\text{cat}}$ .

Design a binary mask  $M^{\text{tab}}$  that enforces the following inductive biases:

1. The **CLS** token can attend to every other token in the row, and every other token can attend to **CLS**.

2. Numeric tokens  $x_i^{\text{num}}$  can *only* attend to the CLS token and to *any* numeric token (including themselves). They cannot attend to categorical tokens.
3. Categorical tokens  $x_i^{\text{cat}}$  can *only* attend to the CLS token and to *any* categorical token (including themselves). They cannot attend to numeric tokens.

Let the token indices be 0 for CLS;  $1, \dots, d_n$  for numeric tokens; and  $d_n + 1, \dots, d_n + d_c$  for categorical tokens.

1. Describe the general block structure of  $M^{\text{tab}}$  for arbitrary  $d_n \geq 1, d_c \geq 1$ . You can use  $\mathbf{1}$  to denote a submatrix of all ones and  $\mathbf{0}$  for a submatrix of all zeros.
2. Provide the specific  $4 \times 4$  binary mask matrix for the case  $(d_n, d_c) = (1, 2)$ . The tokens are  $[\text{CLS}, x_1^{\text{num}}, x_1^{\text{cat}}, x_2^{\text{cat}}]$ .

(d) **Graph neighbourhood mask**

Consider an undirected graph with  $N = 5$  nodes and an adjacency matrix  $A \in \{0, 1\}^{N \times N}$  (where  $A_{ij} = 1$  if node  $i$  and node  $j$  are connected, and  $A_{ij} = 0$  otherwise; assume  $A_{ii} = 0$  for all  $i$ , i.e., no self-loops in  $A$  itself). You want each node to attend only to itself and its direct neighbors in the graph.

1. Provide the general rule to construct an *additive attention mask*  $M^{\text{graph}}$  from the adjacency matrix  $A$ .
2. Given the adjacency matrix  $A$  for  $N = 5$ :

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

Write down the corresponding  $5 \times 5$  additive mask matrix  $M^{\text{graph}}$ .

**Optional** Discuss how your masks (causal, tree, tabular) could be extended or combined if a model processes several independent structures (e.g., multiple sequences, trees, or table rows) *batched* together. Assume the global input sequence is a concatenation of these independent structures, possibly separated by special [SEP] tokens. What is the main principle for masking in such batched scenarios to maintain independence between batch items?

### Exercise 3: Positional Encodings for Special Datatypes

#### Definition: Sinusoidal Positional Encodings

Sinusoidal positional encodings are a type of fixed (non-learned) positional encoding used in Transformer models to inject information about the position of tokens in a sequence. For a token at position ‘pos’ in a sequence and a model of dimension  $d_{\text{model}}$ , the positional encoding vector  $\text{PE}(\text{pos}) \in \mathbb{R}^{d_{\text{model}}}$  is defined as follows:

For each dimension  $j$  from 0 to  $d_{\text{model}} - 1$ :

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \\ \text{PE}(\text{pos}, 2i + 1) &= \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \end{aligned}$$

where  $j = 2i$  or  $j = 2i + 1$ . Here:

- $pos$  is the position of the token in the input sequence (e.g.,  $0, 1, 2, \dots$ ).
- $i$  is the index for the wavelength pair (ranging from 0 to  $\lfloor (d_{\text{model}} - 1)/2 \rfloor$ ).
- $d_{\text{model}}$  is the dimensionality of the embeddings (and thus of the PE vector).

In this exercise you will design positional encodings for data types that are *not* plain 1-D sequences.

Let  $d$  be the model dimension and assume all positional vectors are in  $\mathbb{R}^d$ .

- (a) **Binary tree** Consider the complete binary tree of depth 2 (7 nodes), as shown in Figure 1. The nodes are stored in an array using breadth-first order (BFS): indices  $0, \dots, 6$  correspond to  $[\text{root}, \text{L}, \text{R}, \text{LL}, \text{LR}, \text{RL}, \text{RR}]$ .

1. Propose a *two-dimensional* index ( $\text{depth}(v), \text{offset}(v)$ ) for every node  $v$  and embed it via the 2-D sinusoidal scheme

$$\text{PE}_{\text{tree}}(v) = \text{PE}_{1\text{D}}(\text{depth}(v)) + \text{PE}_{1\text{D}}(\text{offset}(v)),$$

where  $\text{PE}_{1\text{D}}$  is the sine/cosine encoding.

2. Write down the  $7 \times d$  matrix of positional vectors for  $d = 4$ .
3. Generalise your formula to an arbitrary  $b$ -ary tree of depth  $L$ .

- (b) **2-D grid (image patches)** Following the ViT patchification from the lecture, an image of  $H \times W$  pixels is split into  $(H/p) \times (W/p)$  patches. Let the patch at row  $r$ , column  $c$  have the encoding

$$\text{PE}_{2\text{D}}(r, c) = \text{PE}_{1\text{D}}(r) + \text{PE}_{1\text{D}}(c).$$

1. Explain why the row/column information remains disentangled.
2. Show how to include a learned class token [CLS] that attends globally.

- (c) **Multi-variate time series (temporal + variate)** A sensor signal consists of  $V$  variates (channels), and each variate's time series is cut into  $T$  non-overlapping patches (or segments). This yields  $T$  time steps, each having  $V$  associated variate values.

A positional encoding scheme for a token corresponding to variate  $v \in \{0, \dots, V - 1\}$  at time step  $t$  is:

$$\text{PE}_{\text{signal}}(t, v) = \text{PE}_{1\text{D}}(t) + \mathbf{e}_v^{\text{var}},$$

where  $\text{PE}_{1\text{D}}(t)$  is the standard sinusoidal encoding for time step  $t$ , and  $\mathbf{e}_v^{\text{var}} \in \mathbb{R}^d$  is a learned embedding for variate  $v$ .

1. Explain how this additive scheme allows the model to potentially factorise information about "when" an event occurs (temporal aspect) and "on which channel/variante" it occurs.
2. Discuss one potential drawback of this simple additive scheme, especially when dealing with variates that have complex, time-dependent correlations or phase shifts relative to each other.

**Optional** Contrast absolute encodings (as above) with *relative* or *rotary* schemes (slide 104) and argue when they would be preferable for very deep trees or extremely long signals.

### Exercise 4: Compute Attention

Consider a single-head Transformer layer that processes a sequence of three token embeddings. The input matrix  $X \in \mathbb{R}^{3 \times 2}$  contains these embeddings as row vectors (i.e., the first row is the embedding for token 1, the second row for token 2, and the third row for token 3):

$$X = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 2 & 1 \end{bmatrix}.$$

The projection matrices for queries, keys, and values are:

$$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_K = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad W_V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}.$$

Use the *scaled dot-product attention* formula:

$$\text{Att}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

where  $d_k$  is the dimension of the key vectors (and query vectors). The softmax function is applied row-wise to its input matrix. Keep at least four significant digits in intermediate calculations and your final answers.

- (a) **Project queries, keys and values** Compute the matrices  $Q = XW_Q$ ,  $K = XW_K$ , and  $V = XW_V$ . State the value of  $d_k$ .
- (b) **Raw compatibility scores** Compute the  $3 \times 3$  scaled score matrix  $S = \frac{QK^\top}{\sqrt{d_k}}$ .
- (c) **Attention weights** Apply the row-wise softmax to  $S$  to obtain the attention weight matrix  $A = \text{softmax}(S)$ . Verify that each row of  $A$  sums to approximately 1 (accounting for potential rounding).
- (d) **Attention output** Compute the final output matrix  $Z = AV \in \mathbb{R}^{3 \times 2}$ .
- (e) **Causal masking** Repeat parts (b) – (d) using a *causal mask*. This means that before applying the softmax in part (c), you modify the scaled score matrix  $S$  (from part b) such that  $S_{ij} = -\infty$  for all  $j > i$  (i.e., a token cannot attend to subsequent tokens). Let this masked score matrix be  $S_{\text{masked}}$ . Then compute  $A_{\text{causal}} = \text{softmax}(S_{\text{masked}})$  and  $Z_{\text{causal}} = A_{\text{causal}}V$ . Comment on how the output vectors in  $Z_{\text{causal}}$  for tokens 1, 2, and 3 differ from their counterparts in  $Z$  (from part d), and explain why these differences occur.